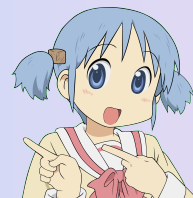


Resuelto de la Práctica N°1 - Programación Funcional del 2^{do} Cuatrimestre 2025

Prácticas de otros cuatrimestres pueden diferir en estrellas y enunciados. Ojo
El siguiente resuelto no tiene ningún fin. Ni garantiza nada!

Si querés hacer alguna corrección, añadir la resolución de un ejercicio o lo que se te ocurra y te cope podés mandar mensaje al grupo de Typst en telegram. Si te gustó como quedó este resuelto podés ir al taller de Typst para aprender a hacerlo!



Currificación y tipos:

1☆ 2☆

Esquemas de recursión:

3☆ 4 5☆ 6☆ 7☆ 8

Otras estructuras de datos:

09☆ 10 11 12☆ 13 14 15☆ 16

Generación infinita:

17 18☆ 19 20☆ 21☆ 22

Parciales, recurs y finales:

2025 2026

Currificación y tipos

1☆ 2☆

Ejercicio 1 ☆

```
- max2 (x, y) | x >= y = x
              | otherwise = y
- normaVectorial (x, y) = sqrt (x^2 + y^2)
- subtract = flip (-)
- predecesor = subtract 1
- evaluarEnCero = \f -> f 0
- dosVeces = \f -> f . f
- flipAll = map flip
- flipRaro = flip flip
```

- I. ¿Cuál es el tipo de cada función? (Suponer que todos los números son de tipo Float).
- II. Indicar cuáles de las funciones anteriores *no* están currificadas. Para cada una de ellas, definir la función currificada correspondiente. Recordar dar el tipo de la función.

Solución:

```
- (Float, Float) -> Float
- (Float, Float) -> Float
- Float -> Float -> Float
- Float -> Float
- (Float -> a) -> a
- (a -> b) -> a -> b
- [(a -> b -> c)] -> [(b -> a -> c)]
- b -> (a -> b -> c) -> a -> c
```

```
max2 :: Float -> Float -> Float
max2 x y | x >= y = x
          | otherwise = y

normaVectorial :: Float -> Float -> Float
normaVectorial x y = sqrt (x^2 + y^2)
```

Ejercicio 2 ☆

- I. Definir la función curry, que dada una función de dos argumentos, devuelve su equivalente currificada.
 - II. Definir la función uncurry, que dada una función currificada de dos argumentos, devuelve su versión no currificada equivalente. Es la inversa de la anterior.
 - III. ¿Se podría definir una función curryN, que tome una función de un número arbitrario de argumentos y devuelva su versión currificada?
- Sugerencia:** pensar cuál sería el tipo de la función.

Solución:

- III. No, no puedo definir el tipo de una función de infinitos argumentos.

```
-- I.
curry :: ((a, b) -> c) -> (a -> b -> c)
curry f = (\x y -> f (x, y))

-- II.
uncurry :: (a -> b -> c) -> ((a, b) -> c)
uncurry f = (\(x, y) -> f x y)
```

Esquemas de recursión

3☆

4

5☆

6☆

7☆

8

Ejercicio 3 ☆

- I. Redefinir usando `foldr` las funciones `sum`, `elem`, `(++)`, `filter` y `map`.
- II. Definir la función `mejorSegún :: (a -> a -> Bool) -> [a] -> a`, que devuelve el máximo elemento de la lista según una función de comparación, utilizando `foldr1`. Por ejemplo, `maximum = mejorSegún (>)`.
- III. Definir la función `sumasParciales :: Num a => [a] -> [a]`, que dada una lista de números devuelve otra de la misma longitud, que tiene en cada posición la suma parcial de los elementos de la lista original desde la cabeza hasta la posición actual. Por ejemplo, `sumasParciales [1,4,-1,0,5] ~> [1,5,4,4,9]`.
- IV. Definir la función `sumaAlt`, que realiza la suma alternada de los elementos de una lista. Es decir, da como resultado: el primer elemento, menos el segundo, más el tercero, menos el cuarto, etc. Usar `foldr`.
- V. Hacer lo mismo que en el punto anterior, pero en sentido inverso (el último elemento menos el anteúltimo, etc.). Pensar qué esquema de recursión conviene usar en este caso.

Solución:

```
-- I.
sum :: Num a => [a] -> a
sum = foldr (+) 0

elem :: Eq a => a -> [a] -> Bool
elem e = foldr ((||) . (==e)) False

(++): [a] -> [a] -> [a]
(++): xs ys = foldr (:) ys xs

filter :: (a -> Bool) -> [a] -> [a]
filter f = foldr (\x rec -> if f x then x:rec else rec) []

map :: (a -> b) -> [a] -> [b]
map f = foldr (:) f []

-- II.
mejorSegún :: (a -> a -> Bool) -> [a] -> a
mejorSegún f = foldr1 (\x rec -> if f rec x then rec else x)

-- III.
sumasParciales :: Num a => [a] -> [a]
sumasParciales xs = foldr (\x rec -> (\n -> (x+n):(rec (x+n)))) (\_ -> []) xs 0

-- IV.
sumaAlt :: Num a => [a] -> a
sumaAlt = foldr (\x rec -> x + (rec * (-1))) 0

-- V.
sumaAltInv :: Num a => [a] -> a
sumaAltInv = foldl (\rec x -> x + (rec * (-1))) 0
```

Ejercicio 4

- I. Definir la función `permutaciones :: [a] -> [[a]]`, que dada una lista devuelve todas sus permutaciones. Se recomienda utilizar `concatMap :: (a -> [b]) -> [a] -> [b]`, y también `take` y `drop`.
- II. Definir la función `partes`, que recibe una lista `L` y devuelve la lista de todas las listas formadas por los mismos elementos de `L`, en su mismo orden de aparición.
Ejemplo: `partes [5, 1, 2] ~> [[], [5], [1], [2], [5, 1], [5, 2], [1, 2], [5, 1, 2]]`
(en algún orden).
- III. Definir la función `prefijos`, que dada una lista, devuelve todos sus prefijos.
Ejemplo: `prefijos [5, 1, 2] ~> [[], [5], [5, 1], [5, 1, 2]]`
- IV. Definir la función `sublistas` que, dada una lista, devuelve todas sus sublistas (listas de elementos que aparecen consecutivos en la lista original).
Ejemplo: `sublistas [5, 1, 2] ~> [[], [5], [1], [2], [5, 1], [1, 2], [5, 1, 2]]`
(en algún orden).

Solución:

```
-- I.
permutaciones :: [a] -> [[a]]
permutaciones = foldr (\x rec -> concatMap (insertV x) rec) [[]]
  where insertV v xs = map (\x -> (take x xs)++v:(drop x xs)) [0..(length xs)]
-- II.
partes :: [a] -> [[a]]
partes = foldr (\x rec -> rec++(map (x:) rec)) [[]]
-- III.
prefijos :: [a] -> [[a]]
prefijos xs = foldl (\rec x -> rec++[take x xs]) [[]] [1..(length xs)]
-- IV.
sublistas :: [a] -> [[a]]
sublistas = foldr (\x rec -> sufijos x ++ rec) [[]] . prefijos
  where sufijos xs = foldr (\x rec -> (drop x xs):rec) [] [0..(length xs - 1)]
```

Ejercicio 5 ★

Considerar las siguientes funciones:

```
elementosEnPosicionesPares :: [a] -> [a]
elementosEnPosicionesPares [] = []
elementosEnPosicionesPares (x:xs) = if null xs
    then [x]
    else x : elementosEnPosicionesPares (tail xs)

entrelazar :: [a] -> [a] -> [a]
entrelazar [] = id
entrelazar (x:xs) = \ys -> if null ys
    then x : entrelazar xs []
    else x : head ys : entrelazar xs (tail ys)
```

Indicar si la recursión utilizada en cada una de ellas es o no estructural. Si lo es, reescribirla utilizando `foldr`. En caso contrario, explicar el motivo.

Solución:

```
entrelazar :: [a] -> [a] -> [a]
entrelazar = foldr (\x rec -> \ys -> if null ys
    then ys ++ rec ys
    else x : head ys : rec (tail ys))
    (\_ -> [])
```

Ejercicio 6 ★

El siguiente esquema captura la recursión primitiva sobre listas.

```
recr :: (a -> [a] -> b -> b) -> b -> [a] -> b
recr _ z [] = z
recr f z (x : xs) = f x xs (recr f z xs)
```

- Definir la función `sacarUna :: Eq a => a -> [a] -> [a]`, que dados un elemento y una lista devuelve el resultado de eliminar de la lista la primera aparición del elemento (si está presente).
- Explicar por qué el esquema de recursión estructural (foldr) no es adecuado para implementar la función `sacarUna` del punto anterior.
- Definir la función `insertarOrdenado :: Ord a => a -> [a] -> [a]` que inserta un elemento en una lista ordenada (de manera creciente), de manera que se preserve el ordenamiento.

Solución:

b) FALTA RESPUESTA B

```
recr :: (a -> [a] -> b -> b) -> b -> [a] -> b
recr _ z [] = z
recr f z (x:xs) = f x xs (recr f z xs)

-- a
sacarUna :: Eq a => a -> [a] -> [a]
sacarUna e = recr (\x xs rec -> if x==e then xs else x:rec) []

-- c
insertarOrdenado :: Ord a => a -> [a] -> [a]
insertarOrdenado e = recr (\x xs rec -> if e < x then e:x:xs else x:rec) [e]
```

Ejercicio 7 ★

Definir las siguientes funciones para trabajar sobre listas, y dar su tipo. Todas ellas deben poder aplicarse a listas *finitas* e *infinitas*.

- `mapPares`, una versión de `map` que toma una función currificada de dos argumentos y una lista de pares.
Pista: recordar `curry` y `uncurry`.
- `armarPares`, que dadas dos listas arma una lista de pares que contiene, en cada posición, el elemento correspondiente a esa posición en cada una de las listas. Si una de las listas es más larga que la otra, ignorar los elementos que sobran (el resultado tendrá la longitud de la lista más corta). Esta función en Haskell se llama `zip`.
Pista: aprovechar la curriificación y utilizar evaluación parcial.
- `mapDoble`, una variante de `mapPares`, que toma una función currificada de dos argumentos y dos listas (de igual longitud), y devuelve una lista de aplicaciones de la función a cada elemento correspondiente de las dos listas. Esta función en Haskell se llama `zipWith`.

Solución:

```
-- I.
mapPares :: (a -> b -> c) -> [(a, b)] -> [c]
mapPares f xs = [f x y | (x, y) <- xs]

-- II.
armarPares :: [a] -> [b] -> [(a,b)]
armarPares xs ys = foldr (\x rec -> \zs ->
  if length zs == 0
  then []
  else (x, head zs):(rec (tail zs))) (\_ -> []) xs ys

-- III.
mapDoble :: (a -> b -> c) -> [a] -> [b] -> [c]
mapDoble f xs ys = foldr (\x rec -> \zs ->
  if length zs == 0
  then []
  else (f x (head zs):(rec (tail zs))) (\_ -> []) xs ys
```

Ejercicio 8

- I. Escribir la función `sumaMat`, que representa la suma de matrices, usando `zipWith`. Representaremos una matriz como la lista de sus filas. Esto quiere decir que cada matriz será una lista finita de listas finitas, todas de la misma longitud, con elementos enteros. Recordamos que la suma de matrices se define como la suma celda a celda. Asumir que las dos matrices a sumar están bien formadas y tienen las mismas dimensiones.
- ```
sumaMat :: [[Int]] -> [[Int]] -> [[Int]]
```
- II. Escribir la función `trasponer`, que, dada una matriz como las del ítem i, devuelva su traspuesta. Es decir, en la posición  $i, j$  del resultado está el contenido de la posición  $j, i$  de la matriz original. Notar que si la entrada es una lista de  $N$  listas, todas de longitud  $M$ , la salida debe tener  $M$  listas, todas de longitud  $N$ .
- ```
trasponer :: [[Int]] -> [[Int]]
```

Solución:

```
-- I.
sumaMat :: [[Int]] -> [[Int]] -> [[Int]]
sumaMat = zipWith (zipWith (+))

-- II.
trasponer :: [[Int]] -> [[Int]]
trasponer m = foldr (zipWith (:)) (map (const []) m) m
```

Otras estructuras de datos

[09 ☆](#)[10](#)[11](#)[12 ☆](#)[13](#)[14](#)[15 ☆](#)[16](#)

Ejercicio 9 ☆

- I. Definir y dar el tipo del esquema de recursión `foldNat` sobre los naturales. Utilizar el tipo `Integer` de Haskell (la función va a estar definida sólo para los enteros mayores o iguales que 0).
- II. Utilizando `foldNat`, definir la función potencia.

Solución:

```
-- I.
foldNat :: (Integer -> b -> b) -> b -> Integer -> b
foldNat _ z 0 = z
foldNat f z n = f n (foldNat f z (n-1))

-- II.
potencia :: Integer -> Integer -> Integer
potencia n = foldNat (\_ rec -> n * rec) 1
```

Ejercicio 10

- I. Definir la función `genLista :: a -> (a -> a) -> Integer -> [a]`, que genera una lista de una cantidad dada de elementos, a partir de un elemento inicial y de una función de incremento entre los elementos de la lista. Dicha función de incremento, dado un elemento de la lista, devuelve el elemento siguiente.
- II. Usando `genLista`, definir la función `desdeHasta`, que dado un par de números (el primero menor que el segundo), devuelve una lista de números consecutivos desde el primero hasta el segundo.

Solución:

```
-- I.
genLista :: a -> (a -> a) -> Integer -> [a]
genLista e f n = foldr (\_ rec -> \y -> y:(rec (f y))) (\y -> []) [1..n] e

-- II.
desdeHasta :: Integer -> Integer -> [Integer]
desdeHasta i j = genLista i (+1) (j-i+1)
```

Ejercicio 11

Definir el esquema de recursión estructural para el siguiente tipo:

```
data Polinomio a = X
                | Cte a
                | Suma (Polinomio a) (Polinomio a)
                | Prod (Polinomio a) (Polinomio a)
```

Luego usar el esquema definido para escribir la función `evaluar :: Num a => a -> Polinomio a -> a` que, dado un número y un polinomio, devuelve el resultado de evaluar el polinomio dado en el número dado.

Solución:

```
data Polinomio a = X
                | Cte a
                | Suma (Polinomio a) (Polinomio a)
                | Prod (Polinomio a) (Polinomio a)

foldPoli :: (b -> b -> b) -> (b -> b -> b) -> (a -> b) -> b -> Polinomio a -> b
foldPoli _ _ v X = v
foldPoli _ _ fc _ (Cte c) = fc c
foldPoli fp fs fc v (Suma x y) = fs (foldPoli fp fs fc v x) (foldPoli fp fs fc v y)
foldPoli fp fs fc v (Prod x y) = fp (foldPoli fp fs fc v x) (foldPoli fp fs fc v y)

evaluar :: Num a => a -> Polinomio a -> a
evaluar v = foldPoli (*) (+) (id) v
```

Ejercicio 12

Considerar el siguiente tipo, que representa a los árboles binarios: `data AB a = Nil | Bin (AB a) a (AB a)`

- I. Usando recursión explícita, definir los esquemas de recursión estructural (`foldAB`) y primitiva (`recAB`), y dar sus tipos.
- II. Definir las funciones `esNil`, `altura` y `cantNodos` (para `esNil` puede utilizarse case en lugar de `foldAB` o `recAB`)
- III. Definir la función `mejorSegún :: (a -> a -> Bool) -> AB a -> a`, análoga a la del ejercicio 3, para árboles. Se recomienda definir una función auxiliar para comparar la raíz con un posible resultado de la recursión para un árbol que puede o no ser `Nil`.
- IV. Definir la función `esABB :: Ord a => AB a -> Bool` que chequea si un árbol es un árbol binario de búsqueda. Recordar que, en un árbol binario de búsqueda, el valor de un nodo es mayor o igual que los valores que aparecen en el subárbol izquierdo y es estrictamente menor que los valores que aparecen en el subárbol derecho.
- V. Justificar la elección de los esquemas de recursión utilizados para los tres puntos anteriores.

Solución:

```
data AB a = Nil | Bin (AB a) a (AB a)

-- I.
foldAB :: (b -> a -> b -> b) -> b -> AB a -> b
foldAB _ v (Nil) = v
foldAB fb v (Bin i x d) = fb (foldAB fb v i) x (foldAB fb v d)

recAB :: (AB a -> b -> a -> AB a -> b -> b) -> b -> AB a -> b
recAB _ v (Nil) = v
recAB fb v (Bin i x d) = fb i (recAB fb v i) x d (recAB fb v d)

-- II.
esNil :: AB a -> Bool
esNil = foldAB (\rec_i _ rec_r -> False && rec_i && rec_r) True

altura :: AB a -> Int
altura = foldAB (\rec_i _ rec_r -> 1 + max rec_i rec_r) 1

cantNodos :: AB a -> Int
cantNodos = foldAB (\rec_i _ rec_r -> 1 + rec_i + rec_r) 1

-- III.
mejorSegun :: (a -> a -> Bool) -> AB a -> a
mejorSegun f (Bin i r d) = foldAB (\fi x fd -> \y -> mejor y (mejor (fi x) (fd x))) (id) (Bin i r d) r
  where mejor x y = if f x y then x else y

-- IV.
esABB :: Ord a => AB a -> Bool
esABB = recAB (\i rec_i x d rec_d -> (valido (>=) x i) && (valido (<) x d)) True
  where valido f x ab = case ab of
      Nil -> True
      (Bin i r d) -> f x r
```

Ejercicio 13

Dado el tipo `AB a` del ejercicio 12:

- I. Definir las funciones `ramas` (caminos desde la raíz hasta las hojas), `cantHojas` y `espejo`.
 - II. Definir la función `mismaEstructura :: AB a -> AB b -> Bool` que, dados dos árboles, indica si éstos tienen la misma forma, independientemente del contenido de sus nodos.
- Pista:** usar evaluación parcial y recordar el ejercicio 7.

Solución:

```
-- I.
ramas :: AB a -> [[a]]
ramas = foldAB (\rec_i x rec_d -> (map (x:) rec_i) ++ (map (x:) rec_d)) [[]]

cantHojas :: AB a -> Int
cantHojas = foldAB (\rec_i _ rec_r -> rec_i + rec_r) 1

espejo :: AB a -> AB a
espejo = foldAB (\rec_i x rec_d -> Bin rec_d x rec_i) Nil

-- II.
mismaEstructura :: AB a -> AB b -> Bool
mismaEstructura = foldAB (\rec_i x rec_d -> \ab -> case ab of
    Nil -> False
    (Bin i r d) -> (rec_i i) && (rec_d d)
    ) (\ab -> case ab of
    Nil -> True
    _ -> False
    )
```

Ejercicio 14

Se desea modelar en Haskell los árboles con información en las hojas (y sólo en ellas). Para esto introduciremos el siguiente tipo:

```
data AIH a = Hoja a | Bin (AIH a) (AIH a)
```

- a) Definir el esquema de recursión estructural `foldAIH` y dar su tipo. Por tratarse del primer esquema de recursión que tenemos para este tipo, se permite usar recursión explícita.
- b) Escribir las funciones `altura :: AIH a -> Integer` y `tamaño :: AIH a -> Integer`. Considerar que la altura de una hoja es 1 y el tamaño de un `AIH` es su cantidad de hojas.

Solución:

```
data AIH a = Hoja a | Bin (AIH a) (AIH a)

-- a)
foldAIH :: (b -> b -> b) -> (a -> b) -> AIH a -> b
foldAIH _ fb (Hoja x) = fb x
foldAIH f fb (Bin x y) = f (foldAIH f fb x) (foldAIH f fb y)

-- b)
altura :: AIH a -> Integer
altura = foldAIH (\x y -> 1 + (max x y)) (const 1)

tamaño :: AIH a -> Integer
tamaño = foldAIH (\x y -> 1 + x + y) (const 1)
```


Ejercicio 15 ★

- I. Definir el tipo `RoseTree` de árboles no vacíos, con una cantidad indeterminada de hijos para cada nodo.
- II. Escribir el esquema de recursión estructural para `RoseTree`. **Importante** escribir primero su tipo.
- III. Usando el esquema definido, escribir las siguientes funciones:
 - a) `hojas`, que dado un `RoseTree`, devuelva una lista con sus hojas ordenadas de izquierda a derecha, según su aparición en el `RoseTree`.
 - b) `distancias`, que dado un `RoseTree`, devuelva las distancias de su raíz a cada una de sus hojas.
 - c) `altura`, que devuelve la altura de un `RoseTree` (la cantidad de nodos de la rama más larga). Si el `RoseTree` es una hoja, se considera que su altura es 1.

Solución:

```
-- I.
data RoseTree a = Hoja a | Tree [RoseTree a]

-- II.
foldRT :: ([b] -> b) -> (a -> b) -> RoseTree a -> b
foldRT _ fb (Hoja x) = fb x
foldRT f fb (Tree xs) = f (map (foldRT f fb) xs)

-- III. a)
hojas :: RoseTree a -> [a]
hojas = foldRT (foldr (++) []) (:[])

-- III. b)
distancias :: RoseTree a -> [Int]
distancias = foldRT (foldr (\x rec -> (map (+1) x)++rec) []) (const [0])

-- III. c)
altura :: RoseTree a -> Int
altura t = 1 + (foldr1 max (distancias t))
```

Ejercicio 16

Se desea representar conjuntos mediante Hashing abierto (*chaining addressing*). El hashing abierto consta de dos funciones: una *función de Hash*, que dado un elemento devuelve un valor entero (el cual se espera que no se repita con frecuencia), y una tabla de Hash, que dado un número entero devuelve los elementos del conjunto a los que la función de Hash asignó dicho número (es decir, la preimagen de la función de Hash para ese número).

Los representaremos en Haskell de la siguiente manera:

```
data HashSet a = Hash (a -> Integer) (Integer -> [a])
```

Por contexto de uso, vamos a suponer que la tabla de Hash es una función total, que devuelve listas vacías para los números que no corresponden a elementos del conjunto. Este es un **invariante** que deberá preservarse en todas las funciones que devuelvan conjuntos.

Definir las siguientes funciones:

- I. vacío :: (a -> Integer) -> HashSet a, que devuelve un conjunto vacío con la función de Hash indicada.
- II. pertenece :: Eq a => a -> HashSet a -> Bool, que indica si un elemento pertenece a un conjunto. Es decir, si se encuentra en la lista obtenida en la tabla de Hash para el número correspondiente a la función de Hash del elemento.

Por ejemplo:

pertenece 5 \$ agregar 1 \$ agregar 2 \$ agregar 1 \$ vacío (flip mod 5) devuelve False.

pertenece 2 \$ agregar 1 \$ agregar 2 \$ agregar 1 \$ vacío (flip mod 5) devuelve True.

- III. agregar :: Eq a => a -> HashSet a -> HashSet a, que agrega un elemento a un conjunto. Si el elemento ya estaba en el conjunto, se debe devolver el conjunto sin modificaciones.
- IV. intersección :: Eq a => HashSet a -> HashSet a -> HashSet a, que, dados dos conjuntos, devuelve un conjunto con la misma función de Hash del primero y con los elementos que pertenecen a ambos conjuntos a la vez.
- V. foldr1 (no relacionada con los conjuntos). Dar el tipo y definir la función foldr1 para listas **sin usar recursión explícita**, recurriendo a alguno de los esquemas de recursión conocidos.
Se recomienda usar la función error :: String -> a para el caso de la lista vacía.

Solución:

```
data HashSet a = Hash (a -> Integer) (Integer -> [a])
-- I.
vacío :: (a -> Integer) -> HashSet a
vacío f = Hash f (\_ -> [])

-- II.
pertenece :: Eq a => a -> HashSet a -> Bool
pertenece v (Hash fa fi) = foldr (\x rec -> v == x || rec) False (fi (fa v))

-- III.
agregar :: Eq a => a -> HashSet a -> HashSet a
agregar v (Hash fa fi) = if pertenece v (Hash fa fi)
    then Hash fa fi
    else Hash fa (\i -> if i == (fa v)
        then v : (fi (fa v))
        else fi i)

-- IV.
interseccion :: Eq a => HashSet a -> HashSet a -> HashSet a
interseccion (Hash fa fi1) (Hash _ fi2) = Hash fa (\i -> foldr (\x rec -> if elem x rec then rec else x:rec) (fi1 i) (fi2 i))

-- V.
foldr1 :: (a -> a -> a) -> [a] -> a
foldr1 f l = if null l
    then error "Lista vacia"
    else recr (\x xs rec -> if null xs then x else f x rec) (head l) l
```

Generación infinita

[17](#)[18★](#)[19](#)[20★](#)[21★](#)[22](#)

Ejercicio 17

¿Cuál es el valor de esta expresión?

```
[ x | x <- [1..3], y <- [x..3], (x + y) 'mod' 3 == 0 ]
```

Solución:

El valor es:

[1, 3]

Ejercicio 18 ★

Definir la lista infinita `paresDeNat :: [(Int, Int)]`, que contenga todos los pares de números naturales: $(0, 0)$, $(0, 1)$, $(1, 0)$, etc.

Solución:

```
paresDeNat :: [(Int, Int)]
paresDeNat = [(x, y) | z <- [0..], x <- [0..z], y <- [0..z], x + y == z]
```

Ejercicio 19

Una tripla pitagórica es una tripla (a, b, c) de enteros positivos tal que $a^2 + b^2 = c^2$.

La siguiente expresión intenta ser una definición de una lista (infinita) de triplas pitagóricas:

```
pitagóricas :: [(Integer, Integer, Integer)]
pitagóricas = [(a, b, c) | a <- [1..], b <- [1..], c <- [1..], a^2 + b^2 == c^2]
```

Explicar por qué esta definición no es útil. Dar una definición mejor.

Solución:

Esta definición no es útil porque a nunca deja de buscar un número que cumpla la condición $a^2 + 1 == 1$.

Una definición mejor sería de la forma:

```
pitagóricas :: [(Integer, Integer, Integer)]
pitagóricas = [(a, b, c) | c <- [1..], a <- [1..c], b <- [1..c], a^2 + b^2 == c^2]
```

Ejercicio 20 ★

Escribir la función `listasQueSuman :: Int -> [[Int]]` que, dado un número natural n , devuelve todas las listas de enteros positivos (es decir, mayores o iguales que 1) cuya suma sea n . Para este ejercicio **se permite usar recursión explícita**. Pensar por qué la recursión utilizada no es estructural. (Este ejercicio no es de generación infinita, pero puede ser útil para otras funciones que generen listas infinitas de listas).

Solución:

```
listasQueSuman :: Int -> [[Int]]
listasQueSuman 0 = [[]]
listasQueSuman n = [x:xs | x <- [1..n], xs <- listasQueSuman(n-x)]
```

Ejercicio 21 ★

Definir en Haskell una lista que contenga todas las listas finitas de enteros positivos (esto es, con elementos mayores o iguales que 1).

Solución:

```
[xs | n <- [1..], xs <- listasQueSuman(n)]
```

Ejercicio 22

Dado el tipo de datos AIH a definido en el ejercicio 14:

- Definir la lista (infinita) de todos los AIH cuyas hojas tienen tipo (). Se recomienda definir una función auxiliar. Para este ejercicio **se permite utilizar recursión explícita**.
- Explicar por qué la recursión utilizada en el punto a) no es estructural.

Solución:

```
-- a)
[x | n <- [1..], x <- arbolesNNodos n]

arbolesNNodos :: Int -> [AIH ()]
arbolesNNodos 1 = [Hoja ()]
arbolesNNodos n = [(Bin x y) | m <- [1..n-1], x <- (arbolesNNodos m), y <- (arbolesNNodos (n-m))]
```

- En arbolesNNodos hago recursion sobre los naturales, de n a 1. Pero en lugar de utilizar el anterior inmediato a n en el caso recursivo, como sería apropiado en recursion estructural, uso todas las combinaciones de entre todos los anteriores a n.



¡FIN DE LA PRÁCTICA 1!

Parciales, recus y finales 2025

1°C-Parcial

final-Diciembre

1°C 2025 - Primer Parcial

Aclaración: en este ejercicio no está permitido utilizar recursión explícita, a menos que se indique lo contrario.

Se define el tipo de datos ABNV a, que representa árboles binarios no vacíos con elementos de tipo a:

```
data ABNV a = Hoja a | Uni a (ABNV a) | Bi (ABNV a) a (ABNV a)
```

Definimos el siguiente árbol para los ejemplos:

```
abnv = Bi (Uni 2 (Hoja 1)) 3 (Bi (Hoja 4) 5 (Uni 2 (Hoja 7)))
```

a) Definir las funciones foldABNV y recABNV, que implementan respectivamente los esquemas de recursión estructural y primitiva para el tipo ABNV a. Solo en este inciso se permite usar recursión explícita.

b) Definir la función elemABNV :: Eq a => a -> ABNV a -> Bool, que indica si un elemento pertenece a un árbol.
Por ejemplo: elemABNV 7 abnv ~> True

c) Definir la función reemplazarUno :: Eq a => a -> a -> ABNV a -> ABNV a que, dados dos elementos x e y y un árbol, devuelve un árbol como el original, pero reemplazando x por y (la primera desde la raíz yendo de izquierda a derecha, en orden de preorder)

Por ejemplo:

```
reemplazarUno 2 5 abnv ~> Bi (Uni 5 (Hoja 1)) 3 (Bi (Hoja 4) 5 (Uni 2 (Hoja 7)))
```

```
reemplazarUno 2 5 (Hoja 1) ~> Hoja 1
```

d) Definir la función nivel :: ABNV a -> Int -> [a], que liste todos los elementos del nivel indicado, de izquierda a derecha, siendo 0 el nivel de la raíz. Si el nivel no existe (ya sea por ser negativo o por superar la altura del árbol), devolver [].

Por ejemplo:

```
nivel abnv 1 ~> [2,5].
```

```
nivel abnv 2 ~> [1,4,2].
```

```
nivel abnv 4 ~> [].
```

Pista: aprovechar la curificación y utilizar evaluación parcial.

Solución:

```
data ABNV a = Hoja a | Uni a (ABNV a) | Bi (ABNV a) a (ABNV a) deriving Show

-- a)
foldABNV :: (a -> b) -> (a -> b -> b) -> (a -> b -> b -> b) -> ABNV a -> b
foldABNV fh _ _ (Hoja x) = fh x
foldABNV fh fu fb (Uni x ab) = fu x (foldABNV fh fu fb ab)
foldABNV fh fu fb (Bi ab1 x ab2) = fb x (foldABNV fh fu fb ab1) (foldABNV fh fu fb ab2)

recABNV :: (a -> b) -> (a -> ABNV a -> b -> b) -> (a -> ABNV a -> ABNV a -> b -> b -> b) -> ABNV a -> b
recABNV fh _ _ (Hoja x) = fh x
recABNV fh fu fb (Uni x ab) = fu x ab (recABNV fh fu fb ab)
recABNV fh fu fb (Bi ab1 x ab2) = fb x ab1 ab2 (recABNV fh fu fb ab1) (recABNV fh fu fb ab2)

-- b)
elemABNV :: Eq a => a -> ABNV a -> Bool
elemABNV e = foldABNV (==e) (\x rec -> e==x || rec) (\x rec1 rec2 -> e==x || rec1 || rec2)

-- c)
reemplazarUno :: Eq a => a -> a -> ABNV a -> ABNV a
reemplazarUno i j = recABNV (\x -> if x==i then Hoja j else Hoja x)
                          (\x ab rec -> if x==i then Uni j ab else Uni x rec)
                          (\x ab1 ab2 rec1 rec2 ->
                            if x==i
                            then Bi ab1 j ab2
                            else (if elemABNV i ab1 then Bi rec1 x ab2 else Bi ab1 x rec2))

-- d)
nivel :: ABNV a -> Int -> [a]
nivel = foldABNV (\x -> (\n -> if n==0 then [x] else []))
               (\x rec -> (\n -> if n==0 then [x] else rec (n-1)))
               (\x rec1 rec2 -> (\n -> if n==0 then [x] else (rec1 (n-1))++(rec2 (n-1))))
```

Sean los siguientes tipos:

```
data Indice = I1 | I2 | I3
data AT a = Hoja a | Nodo (Indice -> AT a)
```

En donde AT a representar el tipo de dato “Árbol Trébol”, dar:

- Dar el tipo e implementar foldAT
- Definir la función altura :: AT a → Int
- Dar el tipo e implementar mapAT que, dada una función, se la aplica a todas las hojas de un árbol trébol.

Todo nodo SIEMPRE tiene 3 árboles trébol asociados.

Solución:

```
data Indice = I1 | I2 | I3
data AT a = Hoja a | Nodo (Indice -> AT a)

-- a.
foldAT :: (a -> b) -> (b -> b -> b -> b) -> AT a -> b
foldAT fh _ (Hoja x) = fh x
foldAT fh ft (Nodo fi) = ft (foldAT fh ft (fi I1)) (foldAT fh ft (fi I2)) (foldAT fh ft (fi I3))

-- b.
altura :: AT a -> Integer
altura = foldAT (const 1) (\i1 i2 i3 -> 1 + max i1 (max i2 i3))

-- c.
mapAT :: (a -> b) -> AT a -> AT b
mapAT f = foldAT (\x -> Hoja (f x)) (\i1 i2 i3 -> Nodo (\i -> case i of
    I1 -> i1
    I2 -> i2
    I3 -> i3))
```

Parciales, recus y finales 2026

final-Febrero

final-Marzo

Final Febrero 2026 (de Christian Cossio Mercado) 🔍

El tipo algebraico `ABBComp` `a` describe a los árboles binarios de búsqueda (ABB) parcialmente comprimidos con elementos de tipo `a` y sin repetidos. Los `ABBComp` son árboles que pueden tener algún subárbol donde, en vez de seguir con una estructura arbórea, sus elementos se incluyen directamente en una lista ordenada (se asume que `a` pertenece a la clase `Ord`).

`ABBComp a = Nil | Comp [a] | Nodo a (ABBComp a) (ABBComp a)`

Por ejemplo, las siguientes expresiones denotan árboles de tipo `ABBComp Int` que denotan los mismos elementos:

- `Comp [0,2,14,22]`
- `Nodo 2 (Nodo 0 Nil Nil) (Comp [14,22])`
- `Nodo 14 (Nodo 2 (Nodo 0 Nil Nil) Nil) (Nodo 22 Nil Nil)`

- Dar el tipo y definir una función `foldABBComp`, que abstraiga el esquema de recursión estructural sobre los *ABB parcialmente comprimidos*.
- Sin usar recursión explícita, dar el tipo y definir una función `mapABBComp` que aplique una función dada a todos los valores de un árbol del tipo indicado anteriormente.
- Dar el tipo y definir `recABBComp` que abstraiga el esquema de recursión primitiva sobre estos árboles.
- Sin usar recursión explícita, dar el tipo y definir la función `ordenado`, que permitirá verificar el invariante de la estructura (i.e., que los elementos de un `ABBComp` estén ordenados).
- Dar el tipo y definir la función `iguales(a1,a2)`, que recibe dos `ABBComp` y devuelve `True` si son iguales al recorrerlos inorder. Se deberá usar alguno de los esquemas de recursión anteriores y justificar por qué se lo usó.

Solución:

```
data ABBComp a = Nil | Comp [a] | Nodo a (ABBComp a) (ABBComp a)

-- a)
foldABBComp :: b -> ([a] -> b) -> (a -> b -> b -> b) -> ABBComp a -> b
foldABBComp z _ _ (Nil) = z
foldABBComp _ fc _ (Comp xs) = fc xs
foldABBComp z fc fn (Nodo x abb1 abb2) = fn x (foldABBComp z fc fn abb1) (foldABBComp z fc fn abb2)

-- b)
mapABBComp :: (a -> b) -> ABBComp a -> ABBComp b
mapABBComp f = foldABBComp (Nil) (\xs -> Comp (map f xs)) (\x rec1 rec2 -> Nodo (f x) rec1 rec2)

-- c)
recABBComp :: b -> ([a] -> b) -> (a -> ABBComp a -> ABBComp a -> b -> b -> b) -> ABBComp a -> b
recABBComp z _ _ (Nil) = z
recABBComp _ fc _ (Comp xs) = fc xs
recABBComp z fc fn (Nodo x abb1 abb2) = fn x abb1 abb2 (recABBComp z fc fn abb1) (recABBComp z fc fn abb2)

-- d)
ordenado :: Ord a => ABBComp a -> Bool
ordenado = recABBComp True
    (\xs -> foldr (\x rec -> \y -> if y <= x then rec x else False) (const True) (tail xs) (head xs))
    (\x abb1 abb2 rec1 rec2 -> (comparar (>=) x abb1) && (comparar (<) x abb2) && rec1 && rec2)
  where comparar f x abb = case abb of
      (Nil) -> True
      (Comp xs) -> f x (last xs)
      (Nodo y _ _) -> f x y

-- e)
iguales :: Eq a => ABBComp a -> ABBComp a -> Bool
iguales abb1 abb2 = (alista abb1) == (alista abb2)
  where alista = foldABBComp [] (id) (\x rec1 rec2 -> rec1++[x]++rec2)
```

El tipo algebraico `AdjAB` a describe a las *adjunciones de árboles binarios* con elementos de tipo `a`. Las `AdjAB` representan listas de árboles binarios, donde cada uno de ellos está al mismo nivel que el otro, que pueden o no tener una raíz en común de acuerdo al constructor usado.

```
data AdjAB a = Raiz a [AB a] | Adj [AB a]
data AB a = Nil | Hoja a | Arb a (AB a) (AB a)
```

Por ejemplo, las siguientes expresiones denotan distintos árboles de tipo `AdjAB Int`:

- `Adj [Nil, Nil, Arb 3 Nil Nil, Hoja 6]`
- `Raiz 5 [Arb 3 (Arb 1 Nil Nil) (Hoja 4), Arb 3 Nil Nil, Nil]`
- `Raiz 10 []`

- Dar el tipo y definir una función `foldAdjAB`, que abstraiga el esquema de recursión estructural sobre las *adjunciones de árboles binarios*, recorriendo todos los árboles por completo.
- Sin usar recursión explícita, dar el tipo y definir una función `mapAdjAB` que aplique una función dada a todos los valores de una adjunción.
- Dar el tipo y definir `recAdjAB` que abstraiga el esquema de recursión primitiva sobre las adjunciones, recorriendo todos los árboles por completo.
- Sin usar recursión explícita, dar el tipo y definir la función `ordenado`, que permita saber si toda la adjunción está ordenada. Así, cada árbol debe estar ordenado como un `ABB` y, adicionalmente, si hubiera raíz común, esta debe ser mayor que la raíz de todos los árboles. En el caso de `Nil`, este árbol está trivialmente ordenado y, además, se lo puede ignorar para comparar con la raíz común.

Solución:

```
data AdjAB a = Raiz a [AB a] | Adj [AB a]
data AB a = Nil | Hoja a | Arb a (AB a) (AB a)

-- a)
foldAB :: b -> (a -> b) -> (a -> b -> b -> b) -> AB a -> b
foldAB z _ _ (Nil) = z
foldAB _ fh _ (Hoja x) = fh x
foldAB z fh fa (Arb x ab1 ab2) = fa x (foldAB z fh fa ab1) (foldAB z fh fa ab2)

foldAdjAB :: b -> (a -> b) -> (a -> b -> b -> b) -> (a -> [b] -> c) -> ([b] -> c) -> AdjAB a -> c
foldAdjAB z fh far fr fad (Raiz x xs) = fr x (map (foldAB z fh far) xs)
foldAdjAB z fh far fr fad (Adj xs) = fad (map (foldAB z fh far) xs)

-- b)
mapAdjAB :: (a -> b) -> AdjAB a -> AdjAB b
mapAdjAB f = foldAdjAB Nil (\x -> Hoja (f x)) (\x rec1 rec2 -> Arb (f x) rec1 rec2)
              (\x recs -> Raiz (f x) recs)
              (\recs -> Adj recs)

-- c)
recAB :: b -> (a -> b) -> (a -> AB a -> AB a -> b -> b -> b) -> AB a -> b
recAB z _ _ (Nil) = z
recAB _ fh _ (Hoja x) = fh x
recAB z fh fa (Arb x ab1 ab2) = fa x ab1 ab2 (recAB z fh fa ab1) (recAB z fh fa ab2)

recAdjAB :: b -> (a -> b) -> (a -> AB a -> AB a -> b -> b -> b)
              -> (a -> [AB a] -> [b] -> c) -> ([AB a] -> [b] -> c) -> AdjAB a -> c
recAdjAB z fh far fr fad (Raiz x xs) = fr x xs (map (recAB z fh far) xs)
recAdjAB z fh far fr fad (Adj xs) = fad xs (map (recAB z fh far) xs)

-- d)
ordenado :: Ord a => AdjAB a -> Bool
ordenado = foldAdjAB (\_ _ -> True) (\x -> \f y -> f y x)
              (\x rec1 rec2 -> \f y -> (f x y) && (rec1 (>=) x) && (rec2 (<) x))
              (\x xs -> and (map (\f -> f (>) x) xs))
              (\xs -> and (map (\f -> f (\_ _ -> True) undefined) xs))
```